Spark SQL Queries and Data Analysis

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

TASK 1 SparkSQL (40%)

Spark SQL is an effective tool for querying both structured and semi-structured data using SQL queries. In Task 3, you will utilize Spark SQL to execute a set of queries on a provided dataset, which can be found in the supplementary files on Blackboard. The queries must be executed using Python and Spark SQL. You may use either Spark's SQL-like API or native SQL commands within the Spark environment to perform the queries. Finally, submit a Jupyter notebook containing the code and clearly displayed query results. Below is the list of queries you need to complete.

Answer:

To handle the large **employees_updated.csv** dataset effectively, I used **Google Colab**.

1. Installing the core engine: **PySpark** (Google Colab)

```
[1]
✓ 10s
!pip install pyspark

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

At first, I installed **PySpark Library** using the **pip** command in **Google Colab**. The O/P shows that it was **already satisfied**, so no extra installation was required.

2. Uploading Dataset: **employees_updated.csv**

```
[2]
✓ 1m
from google.colab import files
uploaded = files.upload()

Choose Files employees_updated.csv
employees_updated.csv(text/csv) - 7105313 bytes, last modified: 4/22/2026 - 100% done
Saving employees_updated.csv to employees_updated.csv
```

Now, I uploaded **employees_updated.csv** into the **Colab** environment. The upload was successful as the file name and size are displayed after running the command.

3. Creating Spark Session: “**SparkSQL**”

```
[3]
✓ 15s
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder \
    .appName("SparkSQL") \
    .getOrCreate()
```

Once the library was installed in Step:1, I established the **SparkSession**. In my case, it works as the bridge between my **Python Script** and **Spark Engine**. I named my session as “**SparkSQL**” to keep my project organized and ensure that all my **SQL Commands** had a dedicated place to ruin itself.

4. Loading Dataset and Displaying Data

```
[4]
✓ 10s
df = spark.read.csv("employees_updated.csv", header=True, inferSchema=True)
df.createOrReplaceTempView("employees")

df.show()
```

EmployeeID	FirstName	LastName	Email	PhoneNumber	HireDate	Commission	DepartmentID	DepartmentName	JobID	JobTitle	salary	LocationID
79237	Joan	Bryan	jamie26@example.org	+44(0)20 74	2017-09-15	0.0	10	HR	5	IT Security Officer	37630.71	78
95133	Colin	Thompson	xallen@example.com	+44(0)11549	2021-11-18	0.0	15	Finance	5	IT Security Officer	37630.71	72
16807	Norman	Lawrence	frank78@example.com	(0117)49601	2017-11-18	0.0	13	Design	2	CTO	64829.3	75
80470	Heather	Lloyd	tracy76@example.net	029 2018 02	2021-08-19	0.0	16	Research	3	CISO	53191.23	80
76153	Abdul	Johnson	bartlettashleigh@...	+44(0)121 4	2019-01-02	0.0	17	Customer Support	10	Financial Analyst	39652.76	74
70894	Thomas	Brookes	butchersara@examp...	+44(0)12149	2019-10-19	0.0	17	Customer Support	6	Analyst	39681.2	74
87528	Jeremy	Wilson	malcolmjones@exam...	0292018019	2018-08-11	0.48	13	Design	1	CEO	88580.99	75
68679	Denise	Ellis	ykirk@example.org	+44(0)15149	2020-10-21	0.0	11	Administration	7	Software Developer	50213.78	50
43815	Clifford	Cameron	danielmarshall@ex...	+44(0)808 1	2023-01-29	0.0	16	Research	10	Financial Analyst	39652.76	80
91066	Billy	Robinson	aimeedean@example...	+4429201826	2020-03-12	0.83	13	Design	10	Financial Analyst	39652.76	75
11996	Douglas	Begum	allan03@example.org	(01632) 960	2017-06-26	0.6	15	Finance	1	CEO	88580.99	72
99002	Cheryl	Barker	ttalbot@example.net	(0116)49601	2023-03-20	0.0	10	HR	6	Analyst	39681.2	78
22645	Sheila	Field	dominicwatson@exa...	(0161) 496	2017-02-26	0.0	16	Research	8	Marketing Manager	54965.17	80
28746	Rosemary	Curtis	gjordan@example.org	+44(0)80815	2019-07-01	0.47	15	Finance	1	CEO	88580.99	72
265	Trevor	Williams	benjamin99@exampl...	+4415149600	2017-05-25	0.0	11	Administration	9	Sales Representative	42860.49	50
51805	Connor	Reynolds	hillgary@example.org	0118 496 03	2018-05-07	0.0	11	Administration	2	CTO	64829.3	50
89500	Charlie	Reid	katieshepherd@exa...	01514960409	2017-12-28	0.0	14	Sales	5	IT Security Officer	37630.71	61
32492	Tina	Marshall	deborah42@example...	(0114) 496	2024-02-04	0.0	11	Administration	4	IT Manager	48609.99	50
66827	Stewart	Cartwright	francisjill@examp...	0141 496 01	2018-02-26	0.0	11	Administration	7	Software Developer	50213.78	50
54357	Clive	Taylor	gwalker@example.net	0808 157 08	2022-06-06	0.0	15	Finance	6	Analyst	39681.2	72

only showing top 20 rows

In my last step for the environment setup, I used **createOrReplaceTempView(“employees”)**. This command helps me to turn dataset into **Virtual Table**.

I also made sure to set **inferSchema=True**, it allows Spark to get the data-types automatically.

1. How many employees were hired in the year 2021?

Answer:

I wanted to filter the data of employees which were hired in the year 2021.I applied the **YEAR ()** function to the **HireDate** Column to get the record of the hired employees in the year 2021.

Result: The query revealed that **6,828** employees were hired in the year 2021.

```
[18]
✓ 0s
#1: Count employees hired in 2021
spark.sql("""
SELECT COUNT(*) AS total_2021
FROM employees
WHERE YEAR(HireDate) = 2021
""").show()

+----------+
|total_2021|
+----------+
|      6828|
+----------+
```

Query 1:

```
spark.sql ("""
SELECT COUNT (*) AS total_2021
FROM employees
WHERE YEAR(HireDate) = 2021
"""). show ()
```

2. Which employees work in a department that is in a physical address with the word ‘close’?

Answer:

I used the **LIKE** operator with wildcards **“%close%”**. I also used **LOWER ()** function to **PhysicalAddress** Column in order to avoid case sensitive issue i.e. **Close** and **close** is treated equally and wasn’t missed.

Result: The result displayed the list of employees with word “close” i.e. **PhysicalAddress – 168 River Close** alongside with their **FirstName** and **LastName**.

```
[19]
✓ 0s
#2: Employees working in address containing 'close'
spark.sql("""
SELECT FirstName, LastName, PhysicalAddress
FROM employees
WHERE LOWER(PhysicalAddress) LIKE '%close%'
""").show()
```

```
+---------+---------+---------------+
|FirstName| LastName|PhysicalAddress|
+---------+---------+---------------+
|   Teresa|   Hughes|168 River Close|
|    Molly|    Kirby|168 River Close|
|  Suzanne|   Thomas|168 River Close|
|    Conor|  Jackson|168 River Close|
|     Tina|    Doyle|168 River Close|
|    Holly| Robinson|168 River Close|
|  Eleanor|   Jordan|168 River Close|
|     Tina|    Mason|168 River Close|
|   Joshua|    Smith|168 River Close|
|   Cheryl|     Shah|168 River Close|
|  Natasha|    Ellis|168 River Close|
|    Conor|   Wright|168 River Close|
|  Stewart|   Turner|168 River Close|
|  Marilyn|  Simpson|168 River Close|
|   Ashley|     Read|168 River Close|
|   Elliot|Whittaker|168 River Close|
|   Stacey|   Coates|168 River Close|
|   Andrea|    Parry|168 River Close|
| Margaret|   Abbott|168 River Close|
| Nicholas|   Harper|168 River Close|
+---------+---------+---------------+
only showing top 20 rows
```

Query 2:

```
spark.sql ("""
SELECT FirstName, LastName, PhysicalAddress
FROM employees
WHERE LOWER(PhysicalAddress) LIKE '%close%'
"""). show ()
```

3. Which job title costs the company more money?

Answer:

I wanted to see which role was the most paid for the company, so I **grouped** everything by **JobTitle** and summed up the salaries. After this, I **sorted** the list from **Highest** to **Lowest**.

Result: The **CEO** position takes the top spot, costing the company a total of roughly **449,548,524**.

```
[20]
✓ 0s
#3: Job title with highest total salary cost
spark.sql("""
SELECT JobTitle, ROUND(SUM(salary), 2) AS total_cost
FROM employees
GROUP BY JobTitle
ORDER BY total_cost DESC
LIMIT 1
""").show()

+--------+--------------+
|JobTitle|    total_cost|
+--------+--------------+
|     CEO|4.4954852425E8|
+--------+--------------+
```

Query 3:

```
spark.sql ("""
SELECT JobTitle, ROUND (SUM (salary), 2) AS total_cost
FROM employees
GROUP BY JobTitle
ORDER BY total_cost DESC
LIMIT 1
"""). show ()
```

4. Calculate the total salary that each employee is going to receive at the end of the month after commission.

Answer:

I calculated the final month payout by adding the **Salary** and **Commission** columns together. I used a **COALESCE** function to treat any empty commission as **Zero**, which prevented the calculation errors.

Result: For example; **Joan Bryan's** total is **37,630.71** similarly **Abdul Johnson's** is **39652.76.**

[21]
0s

```
#4: Total salary after adding commission
spark.sql("""
SELECT FirstName, LastName,
       salary + COALESCE(Commission, 0) AS total_salary
FROM employees
""").show()
```

```
+---------+----------+-----------------+
|FirstName|  LastName|      total_salary|
+---------+----------+-----------------+
|     Joan|     Bryan|          37630.71|
|    Colin|  Thompson|          37630.71|
|   Norman|  Lawrence|           64829.3|
|  Heather|     Lloyd|          53191.23|
|    Abdul|   Johnson|          39652.76|
|   Thomas|   Brookes|           39681.2|
|   Jeremy|    Wilson|          88581.47|
|   Denise|     Ellis|          50213.78|
| Clifford|   Cameron|          39652.76|
|    Billy|  Robinson|39653.590000000004|
|  Douglas|     Begum| 88581.59000000001|
|   Cheryl|    Barker|           39681.2|
|   Sheila|     Field|          54965.17|
| Rosemary|    Curtis|          88581.46|
|   Trevor|  Williams|          42860.49|
|   Connor|  Reynolds|           64829.3|
|  Charlie|      Reid|          37630.71|
|     Tina|  Marshall|          48609.99|
|  Stewart|Cartwright|          50213.78|
|    Clive|    Taylor|           39681.2|
+---------+----------+-----------------+
only showing top 20 rows
```

Query 4:

```
spark.sql ("""
SELECT FirstName, LastName,
      salary + COALESCE (Commission, 0) AS
total_salary
FROM employees
"""). show ()
```

5. In which date of the month does ABC hire most employees?

Answer:

I separated down the **HireDate** to look only at the day of the month using the **DAY ()** function. By **grouping** these days and counting them, I could see which date saw the greatest number of activities for hiring.

Result: The 5th of the month is the most hiring day of the month for **HR**, with the record of 1,733 hires on that day.

```
[22]
✓ 0s
#5: Most common hiring day of the month
spark.sql("""
SELECT DAY(HireDate) AS day, COUNT(*) AS total
FROM employees
GROUP BY day
ORDER BY total DESC
LIMIT 1
""").show()

+---+-----+
|day|total|
+---+-----+
|  5| 1733|
+---+-----+
```

Query 5:

```
spark.sql ("""
SELECT DAY(HireDate) AS day, COUNT (*) AS total
FROM employees
GROUP BY day
ORDER BY total DESC
LIMIT 1
"""). show ()
```

6. Which department has the highest number of employees?

Answer:

I ran a **count** on the total number of employees after **grouping** the data by **DepartmentName**.

Result: The **Finance department** holds largest in the company with total employees of **6,369**.

```
[23]
✓ 0s
#6: Department with highest number of employees
spark.sql("""
SELECT DepartmentName, COUNT(*) AS total
FROM employees
GROUP BY DepartmentName
ORDER BY total DESC
LIMIT 1
""").show()

+--------------+-----+
|DepartmentName|total|
+--------------+-----+
|       Finance| 6369|
+--------------+-----+
```

Query 6:

```
spark.sql ("""
SELECT DepartmentName, COUNT (*) AS total
FROM employees
GROUP BY DepartmentName
ORDER BY total DESC
LIMIT 1
"""). show ()
```

7. What is the average salary for employees in each department?

Answer:

I calculated the mean salary for each department using the **AVG ()** function and **rounded the numbers** to two decimal places for better O/P.

Result: Salaries were fairly balanced among all the departments. For example,

HR at **52,299.61** and **Marketing** at **51,804.88.**

```
[24]
✓ 0s
#7: Average salary in each department
spark.sql("""
SELECT DepartmentName, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY DepartmentName
""").show()

+----------------+----------+
|  DepartmentName|avg_salary|
+----------------+----------+
|           Sales|  52078.27|
|Customer Support|   52085.4|
|              HR|  52299.61|
|         Finance|   52171.5|
|        Research|  51963.85|
|       Marketing|  51804.88|
|          Design|  51942.49|
|  Administration|  51944.01|
+----------------+----------+
```

Query 7:

spark.sql (““““

SELECT DepartmentName, ROUND (AVG (salary),2)

AS avg_salary

FROM employees

GROUP BY DepartmentName

”””). show ()

8. How many employees have been with the company for more than 5 years?

Answer:

To find employees with over 5 years of service in the company, I used **months_between ()** to compare the **current_date ()** against their original **HireDate**, looking for the employees with more than 60 months of service.

Result: A mass total of **29,853** employees has completed their 5 years tenure in this company as a veteran's.

```
[25]
✓ 0s
#8: Employees working for more than 5 years
spark.sql("""
SELECT COUNT(*) AS total
FROM employees
WHERE months_between(current_date(), HireDate) > 60
""").show()

+-----+
|total|
+-----+
|29853|
+-----+
```

Query 8:

spark.sql (““““

SELECT COUNT (*) AS total

FROM employees

WHERE months_between (current_date (), HireDate) > 60

””””). show ()

9. What is the total commission paid to employees in each department?

Answer:

I **grouped** the data by **DepartmentName** and summed all the individual commission payout using the **SUM ()** function.

Result: **HR** is leading with a total commission of **631.88** and after that **Finance** holds the second position with total of **629.72**.

```
[26]
✓ 0s
#9: Total commission paid per department
spark.sql("""
SELECT DepartmentName, SUM(Commission) AS total_commission
FROM employees
GROUP BY DepartmentName
""").show()

+----------------+-----------------+
|  DepartmentName| total_commission|
+----------------+-----------------+
|           Sales|610.2599999999999|
|Customer Support|612.5399999999998|
|              HR|631.8799999999999|
|         Finance|629.7200000000004|
|        Research|610.9700000000008|
|       Marketing|592.1299999999999|
|          Design|614.6599999999996|
|  Administration|609.0200000000001|
+----------------+-----------------+
```

Query 9:

spark.sql (““““

SELECT DepartmentName, SUM (Commission)

AS total_commission

FROM employees

GROUP BY DepartmentName

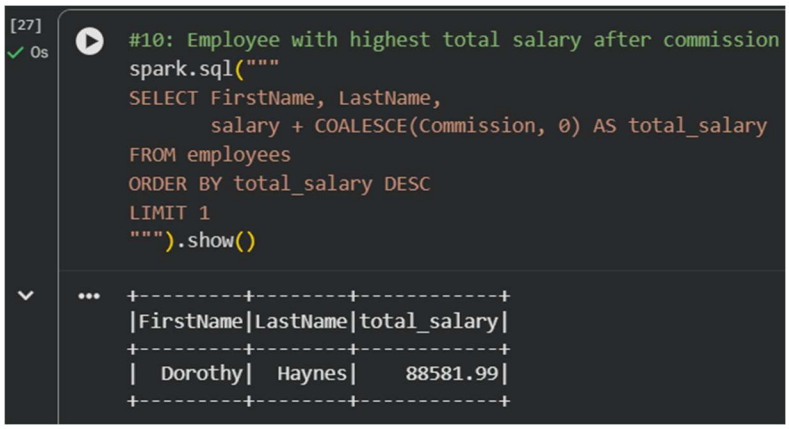
””””). show ()

10. Which employee has the highest salary after commission?

Answer:

I did a final calculation of **Salary + Commission** for every employee, then **sorted** the entire **employees_updated.csv** to find the highest value.

Result: **Dorothy Haynes** is the highest paid employee in the company with the total of **88581.99**.



Query 10:

```
spark.sql ("""""
SELECT FirstName, LastName, salary + COALESCE (Commission, 0)
AS total_salary
FROM employees
ORDER BY total_salary DESC
LIMIT 1
"""""). show ()
```

Tools and Technology Used:

1. Google Colab

I used **Google Colab** to run my **Spark SQL** session and then execute my **python scripts** get the required outputs. **My SQL** commands were executed in Google Colab and all the questions were answered.

2. MS Word

I used **MS Word** to present my answer in more detailed and systemic way. MS Word contains all the **queries**, **outputs**, **screenshots**, etc.